

✓ I. Perform DGE analysis after downloading the dataset from GEO and interpret the results

```
!pip install GEOparse pandas numpy scipy matplotlib seaborn
import GEOparse
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import ttest_ind

print("All libraries imported successfully!")
gse = GEOparse.get_GEO("GSE143272", destdir=".")

print(gse)
```

```

DEBUG:GEOParse:SAMPLE: GSM4255896
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255897
DEBUG:GEOParse:SAMPLE: GSM4255897
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255898
DEBUG:GEOParse:SAMPLE: GSM4255898
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255899
DEBUG:GEOParse:SAMPLE: GSM4255899
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255900
DEBUG:GEOParse:SAMPLE: GSM4255900
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255901
DEBUG:GEOParse:SAMPLE: GSM4255901
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255902
DEBUG:GEOParse:SAMPLE: GSM4255902
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255903
DEBUG:GEOParse:SAMPLE: GSM4255903
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255904
DEBUG:GEOParse:SAMPLE: GSM4255904
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255905
DEBUG:GEOParse:SAMPLE: GSM4255905
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255906
DEBUG:GEOParse:SAMPLE: GSM4255906
16-Aug-2026 07:04:52 DEBUG GEOParse - SAMPLE: GSM4255907
DEBUG:GEOParse:SAMPLE: GSM4255907
<SERIES: GSE143272 - 142 SAMPLES, 1 d(s)>

```

```

print("Title:")
print(gse.metadata["title"][0])

print("\nSummary:")
print(gse.metadata["summary"][0])

print("\nOverall Design:")
print(gse.metadata["overall_design"][0])
sample = list(gse.gsms.values())[0]

print(sample.metadata.keys())
for key, value in sample.metadata.items():
    print(f"{key}: {value}")

```

```

Title:
Peripheral blood expression profiles of patients with epilepsy receiving and not

Summary:
The aim here was to identify mRNA expression biomarkers associated with the dise

Overall Design:
Total RNA obtained from peripheral blood samples of 34 drug-naïve patients with
dict_keys(['title', 'geo_accession', 'status', 'submission_date', 'last_update_d
title: ['Healthy Control-21']
geo_accession: ['GSM4255766']
status: ['Public on Jan 08 2020']
submission_date: ['Jan 07 2020']
last_update_date: ['Jan 08 2020']

```

```

type: ['RNA']
channel_count: ['1']
source_name_ch1: ['Whole blood']
organism_ch1: ['Homo sapiens']
taxid_ch1: ['9606']
characteristics_ch1: ['age (in years): 26', 'Sex: Female', 'epilepsy type: -', '
molecule_ch1: ['total RNA']
extract_protocol_ch1: ['RNA was extracted with Trizol reagent, followed by clean
label_ch1: ['biotin']
label_protocol_ch1: ['Biotinylated cRNA samples were prepared with the Ambion Me
hyb_protocol: ['Standard Illumina hybridization protocol']
scan_protocol: ['Standard Illumina scanning protocol']
description: ['Healthy Control']
data_processing: ["The raw intensity files were extracted from GenomeStudio and
platform_id: ['GPL10558']
contact_name: ['Ritushree,,Kukreti']
contact_email: ['ritus@igib.in']
contact_laboratory: ['403B']
contact_department: ['Genomics and Molecular Medicine Unit']
contact_institute: ['CSIR-IGIB']
contact_address: ['CSIR-IGIB, Mall road, Opp. Jubilee Hall, University of Delhi']
contact_city: ['New Delhi']
contact_state: ['Delhi']
contact_zip/postal_code: ['110007']
contact_country: ['India']
supplementary_file: ['NONE']
series_id: ['GSE143272']
data_row_count: ['13165']

```

```

sample_info = []

for gsm_name, gsm in gse.gsms.items():
    title = gsm.metadata.get("title", [""])[0]
    description = gsm.metadata.get("description", [""])[0]

    # Extract 'Sex', 'age (in years)', 'ethnicity', 'epilepsy type', 'treatment
    sex = "Unknown"
    age = "Unknown"
    ethnicity = "Unknown"
    epilepsy_type = "Unknown"
    treatment = "Unknown"
    drug_response = "Unknown"

    characteristics = gsm.metadata.get("characteristics_ch1", [])
    for char in characteristics:
        if "Sex:" in char:
            sex = char.split("Sex: ")[1]
        if "age (in years):" in char:
            age = char.split("age (in years): ")[1]
        if "ethnicity:" in char:
            ethnicity = char.split("ethnicity: ")[1]
        if "epilepsy type:" in char:

```

```

        epilepsy_type = char.split("epilepsy type: ")[1]
    if "treatment:" in char:
        treatment = char.split("treatment: ")[1]
    if "drug response:" in char:
        drug_response = char.split("drug response: ")[1]

    sample_info.append({
        "Sample": gsm_name,
        "Title": title,
        "Description": description,
        "Sex": sex,
        "age (in years)": age,
        "ethnicity": ethnicity,
        "epilepsy type": epilepsy_type,
        "treatment": treatment,
        "drug response": drug_response
    })

sample_df = pd.DataFrame(sample_info)

sample_df.head(10)
sample_df["Description"].value_counts()

```

	count
Description	
Healthy Control	51
Drug-naïve	34
Valproate Responder	16
Carbamazepine Responder	10
Valproate Non-responder	9
Carbamazepine Non-responder	9
Phenytoin Responder	7
Phenytoin Non-responder	6

dtype: int64

```

sample_df["Group"] = sample_df["Description"].replace({
    "Healthy Control": "Control",
    "Drug-naïve": "Epilepsy"
})
sample_df = sample_df[sample_df["Group"].isin(["Control", "Epilepsy"])]
# Count samples in each group
sample_df["Description"].value_counts().to_frame("count")
import matplotlib.pyplot as plt

```

```

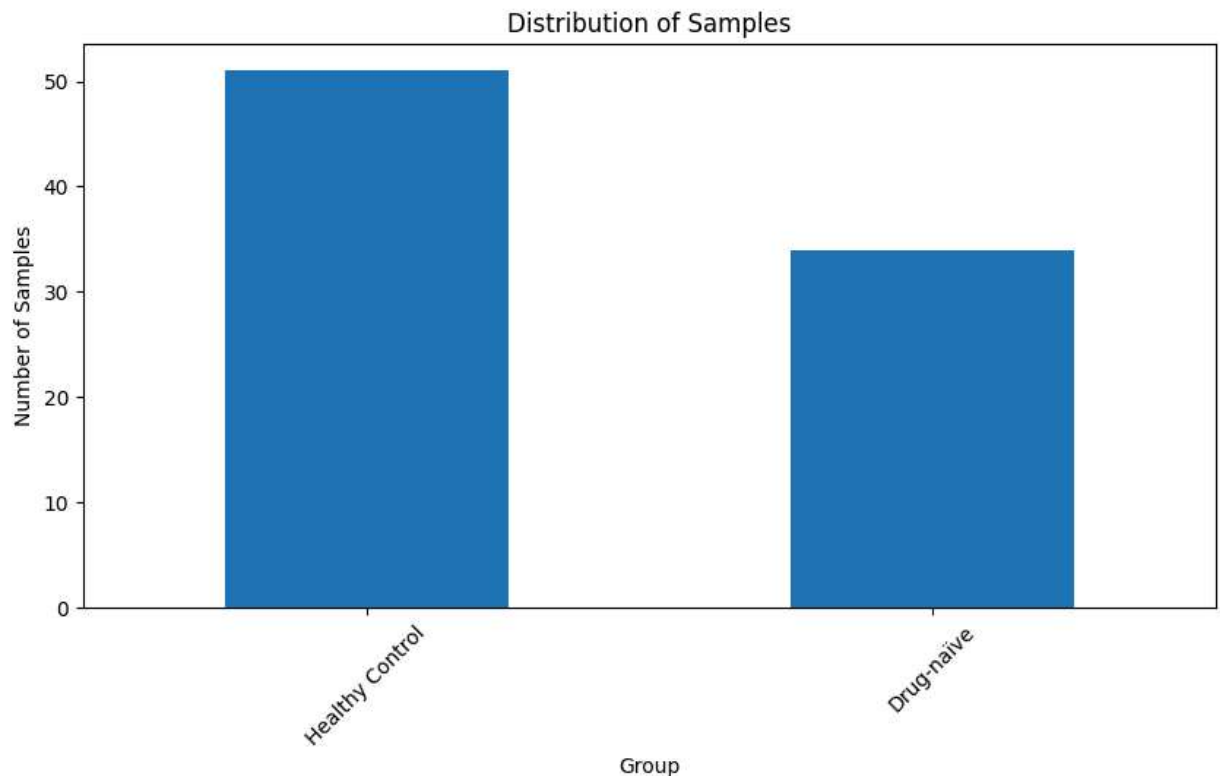
sample_df["Description"].value_counts().plot(
    kind="bar",
    figsize=(10,5)
)

plt.title("Distribution of Samples")
plt.xlabel("Group")
plt.ylabel("Number of Samples")
plt.xticks(rotation=45)
plt.show()
controls = sample_df[sample_df["Description"]=="Healthy Control"]

patients = sample_df[sample_df["Description"]!="Healthy Control"]

print("Controls:", len(controls))
print("Patients:", len(patients))
sample_df["Sex"].value_counts()
pd.crosstab(sample_df["Description"], sample_df["Sex"])
sample_df["age (in years)"] = sample_df["age (in years)"].astype(float)
sample_df.groupby("Description")["age (in years)"].describe()
sample_df.groupby("Description")["age (in years)"].mean().sort_values()
sample_df["ethnicity"].value_counts()
sample_df["epilepsy type"].value_counts()
sample_df["drug response"].value_counts()
sample_df["treatment"].value_counts()
pd.crosstab(
    sample_df["treatment"],
    sample_df["drug response"]
)
pd.crosstab(
    sample_df["epilepsy type"],
    sample_df["treatment"]
)
sample_df.to_csv("GSE143272_metadata.csv", index=False)
expression = pd.DataFrame({
    gsm_name: gsm.table.set_index("ID_REF")["VALUE"]
    for gsm_name, gsm in gse.gsms.items()
})
expression.shape
expression.to_csv("GSE143272_expression_matrix.csv")
expression.head()
expression.describe()

```



Controls: 51

Patients: 34

	GSM4255766	GSM4255767	GSM4255768	GSM4255769	GSM4255770	GSM4255771
count	13165.000000	13165.000000	13165.000000	13165.000000	13165.000000	13165.000000
mean	6.633764	6.675420	6.642817	6.628014	6.646827	6.633764
std	1.975923	2.010763	1.978294	1.965690	1.960679	1.975923
min	2.629514	2.391777	2.464689	2.007298	2.195687	2.629514
25%	5.095004	5.113964	5.099637	5.097993	5.131680	5.095004
50%	6.147312	6.265253	6.168839	6.163985	6.229061	6.147312

```
print(len(sample_df))
print(sample_df["Description"].value_counts())
```

```
85
Description
Healthy Control    51
Drug-naïve        34
Name: count, dtype: int64
```

```
# Select only Healthy Controls and Drug-naïve samples

selected_samples = sample_df[
    sample_df["Description"].isin(["Healthy Control", "Drug-naïve"])
].copy()
```

```

print(selected_samples["Description"].value_counts())

control_ids = selected_samples[
    selected_samples["Description"]=="Healthy Control"
]["Sample"].tolist()

patient_ids = selected_samples[
    selected_samples["Description"]=="Drug-naïve"
]["Sample"].tolist()

print(len(control_ids))
print(len(patient_ids))

expr = expression[control_ids + patient_ids]

expr.shape

```

```

Description
Healthy Control    51
Drug-naïve         34
Name: count, dtype: int64
51
34
(13165, 85)

```

```

!pip install statsmodels

results = []

for gene in expr.index:

    control = expr.loc[gene, control_ids].astype(float)

    patient = expr.loc[gene, patient_ids].astype(float)

    t_stat, p_value = ttest_ind(patient,
                                control,
                                equal_var=False)

    logFC = patient.mean() - control.mean()

    results.append([gene, logFC, p_value])

import pandas as pd

deg = pd.DataFrame(
    results,
    columns=["Gene", "logFC", "P-value"]
)

deg.head()

```

```
from statsmodels.stats.multitest import multipletests

deg["Adj_P"] = multipletests(
    deg["P-value"],
    method="fdr_bh"
)[1]
significant = deg[
    (deg["Adj_P"] < 0.05) &
    (abs(deg["logFC"]) > 1)
]

print(significant.shape)

significant.head()
significant = deg[
    deg["Adj_P"] < 0.05
]

print(significant.shape)
significant = significant.sort_values(
    by="Adj_P"
)

significant.head(20)

significant.to_csv(
    "Differentially_Expressed_Genes.csv",
    index=False
)
from google.colab import files

files.download("Differentially_Expressed_Genes.csv")
import numpy as np
import matplotlib.pyplot as plt

deg["minusLog10P"] = -np.log10(deg["Adj_P"])

plt.figure(figsize=(8,6))

plt.scatter(
    deg["logFC"],
    deg["minusLog10P"],
    alpha=0.5
)

plt.xlabel("log Fold Change")
plt.ylabel("-log10(FDR)")
plt.title("Volcano Plot")

plt.axvline(1, color='red', linestyle='--')
plt.axvline(-1, color='red', linestyle='--')
```



```
plt.axhline(-np.log10(0.05), color= 'blue' , linestyle='--')

plt.tight_layout()
plt.savefig("volcano_plot.png", dpi=300, bbox_inches="tight")
plt.show()

import seaborn as sns

top50 = significant.head(50)["Gene"]

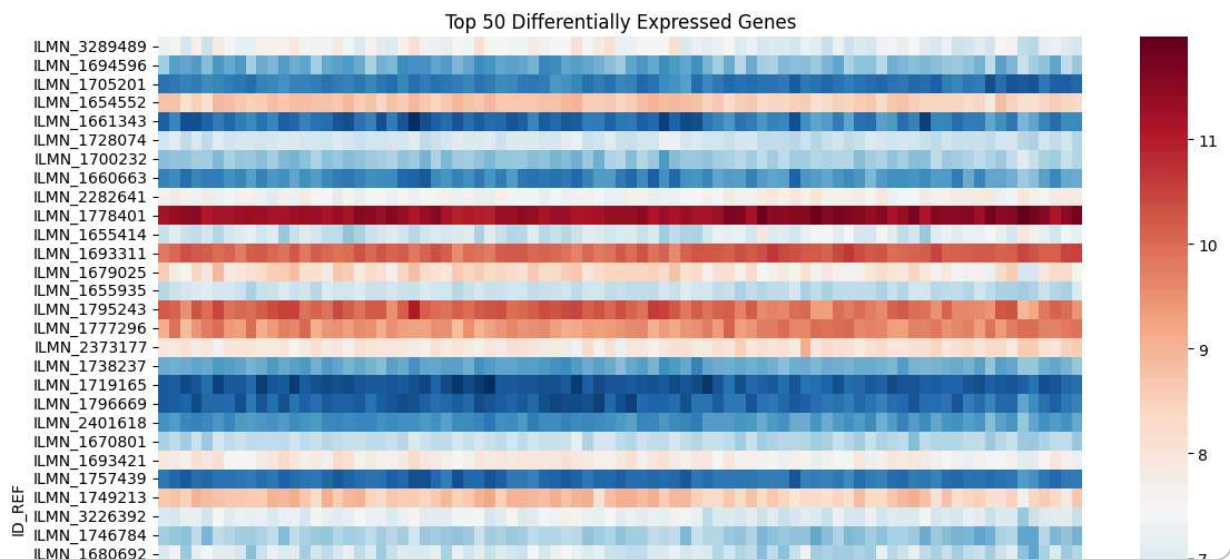
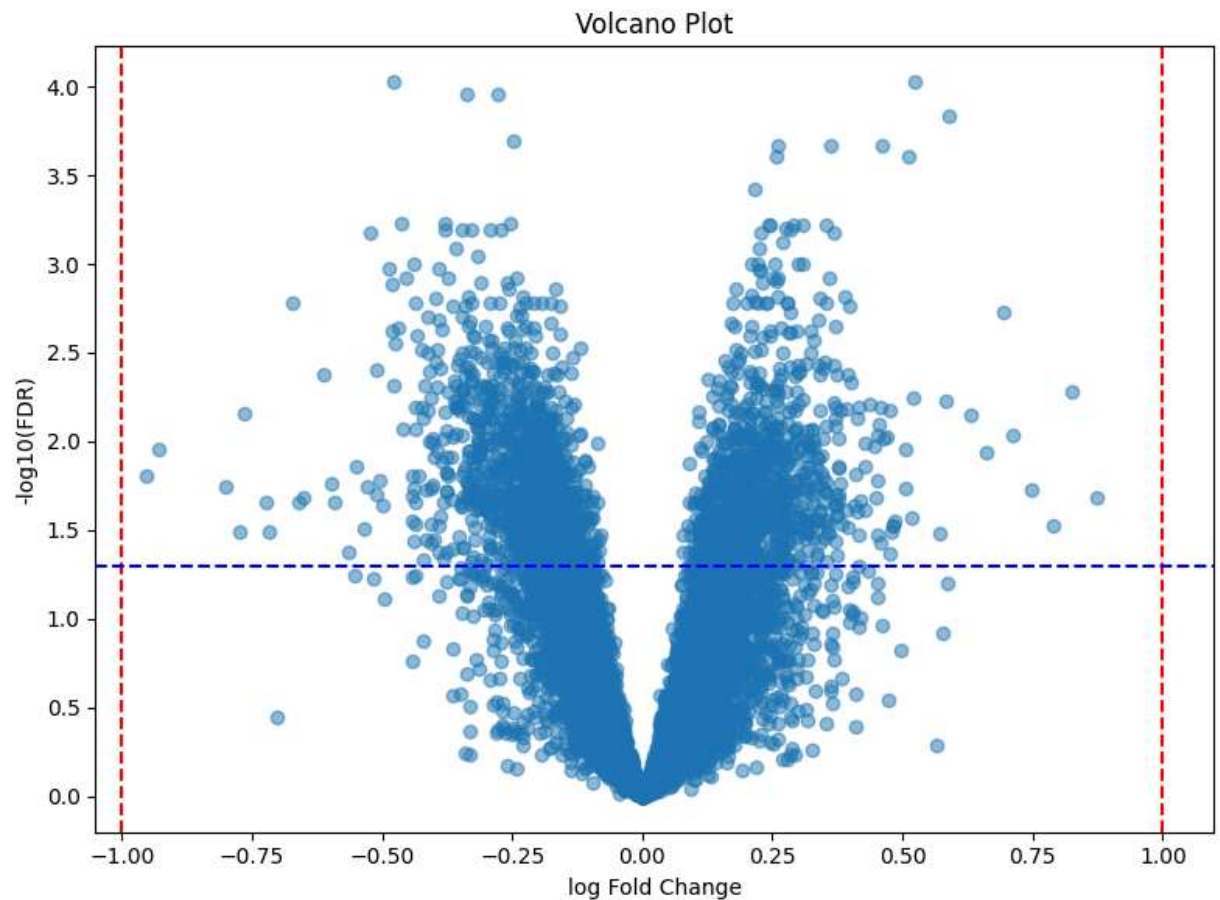
heatmap_data = expr.loc[top50]

plt.figure(figsize=(12,10))

sns.heatmap(
    heatmap_data,
    cmap="RdBu_r"
)

plt.title("Top 50 Differentially Expressed Genes")
plt.tight_layout()
plt.savefig("heatmap.png", dpi=300, bbox_inches="tight")
plt.show()
```

(0, 4)
(2322, 4)



✓ STEPS:

1. Library Installation and Data Loading: Essential libraries like GEOparse, pandas, numpy, scipy, matplotlib, and seaborn are installed and imported. The GSE143272 dataset is then downloaded and loaded using GEOparse.get_GEO.
2. Metadata Extraction and Cleaning:

- Key metadata (title, summary, overall design) and sample-specific metadata are extracted from the gse object.
- A sample_df DataFrame is created to store sample information, including 'Sample', 'Title', 'Description', 'Sex', 'age (in years)', 'ethnicity', 'epilepsy type', 'treatment', and 'drug response', all extracted from the characteristics_ch1 field.

3. Sample Grouping and Initial Analysis:

- A 'Group' column is added to sample_df, categorizing 'Healthy Control' as 'Control' and 'Drug-naïve' as 'Epilepsy'.
- The sample_df is filtered to include only 'Control' and 'Epilepsy' groups.
- Various metadata distributions (e.g., sample counts, sex, age, ethnicity, epilepsy type, drug response, treatment) are analyzed and visualized.
- The expression DataFrame is created from the raw gene expression data, indexed by gene IDs.

4. Filtering Expression Data for DGE Analysis:

- sample_df is further filtered to select only 'Healthy Control' and 'Drug-naïve' samples.
- Control and patient sample IDs are extracted.
- The expression DataFrame (expr) is subsetted to include only these selected control and patient samples.

5. Differential Gene Expression (DGE) Analysis:

- A loop iterates through each gene in the expr DataFrame.
- For each gene, t-tests are performed to compare expression levels between patient and control groups using scipy.stats.ttest_ind.
- Log Fold Change (logFC) is calculated as the difference in mean expression between patient and control groups.
- Results (Gene, logFC, P-value) are collected into a DataFrame deg.
- Adjusted p-values (Adj_P) are calculated using the Benjamini-Hochberg (FDR) method via statsmodels.stats.multitest.multipletests to correct for multiple comparisons.
- Significant genes are identified based on $\text{Adj_P} < 0.05$ and $|\log\text{FC}| > 1$ (initially) and then only $\text{Adj_P} < 0.05$.
- The significant genes are sorted by adjusted p-value and saved to a CSV file (Differentially_Expressed_Genes.csv).

6. Visualization of DGE Results:

- A Volcano Plot is generated to visualize logFC against $-\log_{10}(\text{Adjusted P-value})$, highlighting genes with significant expression changes.
- A Heatmap is generated for the top 50 differentially expressed genes, showing their expression patterns across the selected control and patient samples.